EXPERIMENT 1: ICMP PACKET ANALYSIS (PING)

1.1 Aim

To capture and analyse Internet Control Message Protocol (ICMP) packets generated by the ping command, to compute the Round Trip Time (RTT), to examine frame, IP and ICMP lengths, and to observe IP fragmentation when large ICMP packets are sent.

1.2 Requirements

- Wireshark packet analyser installed on the source host
- Two hosts connected in the same LAN; the destination host has the IP address 192.168.1.20
- Command Prompt (Windows) with administrative privileges

1.3 Background

The ping command uses the ICMP protocol (IP protocol number 1) to test the reachability of a host. The source sends an ICMP Echo Request (Type 8, Code 0) and the destination answers with an ICMP Echo Reply (Type 0, Code 0). The time between sending the request and receiving the reply is the Round Trip Time (RTT).

1.4 Part A: Information Collected from an ICMP Packet

Procedure

1. Open Wireshark, select the active network interface and start the capture.
2. Apply the display filter: icmp
3. In the Command Prompt, execute: ping 192.168.1.20
4. Stop the capture after the replies are received.
5. Select the first Echo (ping) request packet. Expand Frame, Internet Protocol Version 4 and Internet Control Message Protocol in the packet details pane and record the values.
6. Select the corresponding Echo (ping) reply packet and record the values in the same way.

Table 1.1: ICMP packet information

Field	Request	Reply
Source IP		
Destination IP		
ICMP type		
Sequence number		
Timestamp		

1.5 Part B: Calculation of RTT

Procedure

1. Start a new capture with the display filter: icmp
2. Execute: ping -n 10 192.168.1.20 (this generates 10 ICMP request/reply transactions).
3. Set the time format to Seconds Since Beginning of Capture (View > Time Display Format).
4. For each transaction, note the frame number and time of the request and of the matching reply (the reply has the same sequence number).
5. Compute the RTT for each transaction and the average RTT.

RTT = Time of Echo Reply − Time of Echo Request

Average RTT = (Sum of RTT of all 10 transactions) / 10

Table 1.2: RTT of 10 ICMP transactions

No.	Seq. No.	Request Frame	Request Time (s)	Reply Frame	Reply Time (s)	RTT (ms)
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						

Sum of RTT = _____ ms Average RTT = _____ ms

Verify the computed average against the Minimum / Maximum / Average values reported by the ping command: _____.

1.6 Part C: Frame Length, IP Packet Length and ICMP Payload Size

Procedure

1. Select an Echo Request packet. Note the Frame length (Frame > Frame Length, in bytes).
2. Expand Internet Protocol Version 4 and note the Header Length and Total Length.
3. Expand Internet Control Message Protocol and note the size of the Data field (ICMP payload).
4. Repeat the observation for the matching Echo Reply packet.

Table 1.3: Packet lengths for the default ping

Parameter	Echo Request	Echo Reply
Total frame length (bytes)		
Ethernet header (bytes)		
IP header length (bytes)		
IP packet (total) length (bytes)		
ICMP header (bytes)		
ICMP payload size (bytes)		

Relationship to verify:

- IP total length = IP header length + ICMP header + ICMP payload
- Frame length = Ethernet header (14 bytes) + IP total length

1.7 Part D: Large ICMP Packets and IP Fragmentation

Caution: Run this part only on the laboratory network and keep the packet count small (use -n 4). Sending large ICMP packets in a continuous manner is a flooding attack and must not be done on any network without authorisation.

Procedure

1. Start the capture and run the default ping: ping -n 4 192.168.1.20
2. Run: ping -l 1400 -n 4 192.168.1.20
3. Run: ping -l 5400 -n 4 192.168.1.20
4. To list the fragments, apply the display filter:

   icmp and ip.flags.mf == 1 || ip.frag_offset > 0
5. For each case, select the packets and record the values in Table 1.4. For fragmented packets, record the values of every fragment (use additional rows or space if required).

Note: The option -l sets the data size in bytes in Windows. In Linux, the equivalent option is -s.

Table 1.4: Effect of ICMP data size on fragmentation

Parameter	Default packet size	-l 1400	-l 5400
ICMP data size (bytes)			
IP total length (bytes)			
Ethernet frame size (bytes)			
Number of IP fragments			
Fragment offset			
More Fragments flag			
IP Identification			

Formulas for verification (Ethernet MTU = 1500 bytes, IP header = 20 bytes, ICMP header = 8 bytes):

- IP total length of an unfragmented packet = 20 + 8 + ICMP data size
- Maximum IP payload per fragment = 1500 − 20 = 1480 bytes (a multiple of 8)
- Number of fragments = ceiling of (8 + ICMP data size) / 1480
- Fragment offset (in the IP header) = (bytes already sent in earlier fragments) / 8
- All fragments of one packet carry the same IP Identification; the More Fragments flag is 1 for every fragment except the last

1.8 Questions

1. Which protocol is used by the ping command and at which layer does it operate?
2. Why does ping not use TCP or UDP port numbers?
3. Why was the -l 5400 packet fragmented while the -l 1400 packet was not?
4. How does the receiving host reassemble the fragments?
5. What happens to the entire datagram if one fragment is lost?

EXPERIMENT 2: TCP THREE-WAY HANDSHAKE

2.1 Aim

To capture a TCP connection establishment to a web server on port 80 and to prove the three-way handshake from the SYN, SYN/ACK and ACK segments.

2.2 Background

TCP establishes a reliable connection using three segments. The client sends a SYN with its initial sequence number (x). The server replies with a SYN/ACK carrying its own initial sequence number (y) and the acknowledgement number x + 1. The client completes the handshake with an ACK having the acknowledgement number y + 1.

2.3 Procedure

1. Start the Wireshark capture on the active interface.
2. In the Command Prompt, run: curl http://192.168.1.20 (the web server listens on port 80).
3. Stop the capture after the page is received.
4. Make sure relative sequence numbers are enabled (Edit > Preferences > Protocols > TCP > Relative sequence numbers). Also note the raw values by disabling it, if required.
5. Apply the following display filters one by one and identify each packet of the handshake:

   tcp.flags.syn == 1 && tcp.flags.ack == 0 **(SYN)**

   tcp.flags.syn == 1 && tcp.flags.ack == 1 **(SYN/ACK)**

   tcp.flags.syn == 0 && tcp.flags.ack == 1 **(ACK)**

To restrict the capture to the web traffic only, the filter tcp.port == 80 may also be used.

6. Record the observations in Table 2.1.

Table 2.1: TCP three-way handshake

Packet	Frame No.	Seq	ACK	Flags
SYN				
SYN/ACK				
ACK				

2.4 Questions

1. Why does a SYN segment consume one sequence number even though it carries no data?
2. What is the purpose of the initial sequence number?

3. Which TCP options were negotiated during the handshake?
4. How many segments are exchanged for connection termination, and why?

EXPERIMENT 3: ANALYSIS OF A FAILED TCP TRANSACTION

3.1 Aim

To capture a failed TCP transaction on port 80 and to analyse retransmissions, duplicate acknowledgements, sequence and acknowledgement numbers, throughput, RTT, TCP window and connection recovery.

3.2 Procedure

1. Create a failure condition on the destination host (192.168.1.20), for example: stop the web service, block port 80 using the firewall, or introduce packet loss on the link.
2. Start the Wireshark capture.
3. In the Command Prompt, run:

   curl http://192.168.1.20
4. Stop the capture after the command ends or times out.
5. Apply the display filter tcp.port == 80 to analyse the port 80 traffic.
6. Use the filters given in each section below and record the observations.

Note: If the port is closed, the destination replies with a RST segment (connection refused). If the packets are silently dropped, the client retransmits the SYN segment at increasing intervals until it gives up.

Table 3.0: Summary of the capture

Item	Observation
Total packets with tcp.port == 80	
Type of failure observed	
Total capture duration (s)	

3.3 TCP Retransmissions

Filter: tcp.analysis.retransmission. A retransmitted segment carries the same sequence number as the original segment.

Table 3.1: TCP retransmissions

Retx. Frame	Seq No.	Original Frame	Time of Original (s)	Time of Retx. (s)	Interval (s)

3.4 Duplicate ACKs

Filter: tcp.analysis.duplicate_ack. A duplicate ACK repeats the same acknowledgement number.

Table 3.2: Duplicate acknowledgements

Frame No.	ACK Number	Dup ACK No.	Fast Retransmission Frame (if any)

3.5 Sequence Numbers

Record the sequence numbers of the data segments sent by the client and the server. Next Sequence Number = Seq + TCP payload length.

Table 3.3: Sequence number analysis

Frame No.	Source	Seq	Length	Next Seq

3.6 ACK Numbers

Bytes acknowledged = current ACK number − previous ACK number.

Table 3.4: Acknowledgement number analysis

Frame No.	Source	ACK	Bytes Acknowledged

3.7 Throughput

Use Statistics > Conversations > TCP (or Statistics > I/O Graph) to obtain the bytes transferred and the duration.

Throughput (bits/s) = (Total bytes transferred × 8) / Duration (s)

Table 3.5: Throughput

Parameter	Value
Total bytes (A to B)	
Total bytes (B to A)	
Duration (s)	
Throughput (bits/s)	

3.8 RTT

Filter: tcp.analysis.ack_rtt. The initial RTT is the time between the SYN and the SYN/ACK (tcp.analysis.initial_rtt). Statistics > TCP Stream Graphs > Round Trip Time may be used to plot the RTT.

Table 3.6: RTT

Parameter	Value (ms)
Initial RTT (handshake)	
Minimum RTT	
Maximum RTT	
Average RTT	

3.9 TCP Window

Examine the Window Size value and Calculated Window Size in the TCP header, together with the window scale factor negotiated in the SYN segments. Filters: tcp.analysis.zero_window and tcp.analysis.window_full.

Table 3.7: TCP window

Frame No.	Source	Window Size	Scale Factor	Calculated Window

3.10 Connection Recovery

Identify how the connection behaved after the failure: whether the retransmission succeeded and the transfer continued, or whether the connection was reset or timed out.

Table 3.8: Connection recovery events

Event	Frame No.	Time (s)	Description
First failure indication			
Retransmission / Dup ACK			
Recovery / RST / FIN			
Final state of the connection			

3.11 Questions

1. How does TCP detect that a segment has been lost?
2. How does the retransmission interval change between successive retransmissions, and why?
3. What is the difference between a timeout retransmission and a fast retransmission?
4. How does the TCP window size affect the throughput?
5. What is the difference between a connection refused (RST) and a connection timeout?

RESULT

Thus the ICMP and TCP packets were captured and analysed using Wireshark, and the RTT, packet lengths, IP fragmentation, the TCP three-way handshake and the behaviour of a failed TCP transaction were observed and verified.

Staff Signature: _____ Marks: _____